IMPLEMENTACIÓN DE MONOLITOS ESCALABLES EN AWS

Módulo 6: Escalabilidad, Alta Disponibilidad y Monitoreo

Autor: Jimmy Agüero *Estudiante Arquitecto Cloud*

Institución: Alkemy / AWS Academy

1. Situación Inicial

La aplicación de una empresa de servicios digitales funciona en un único servidor on-premise, lo que ocasiona caídas en picos de tráfico y altos costos de mantenimiento. La organización desea migrar la aplicación monolítica a la nube, manteniendo el enfoque de monolito, pero asegurando escalabilidad automática, alta disponibilidad y monitoreo.

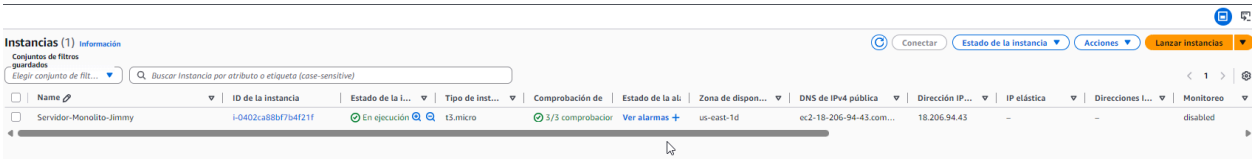
Una empresa con un servidor on-premise que sufre caídas en picos de tráfico y altos costos. El objetivo es migrar a una arquitectura escalable y de alta disponibilidad en AWS

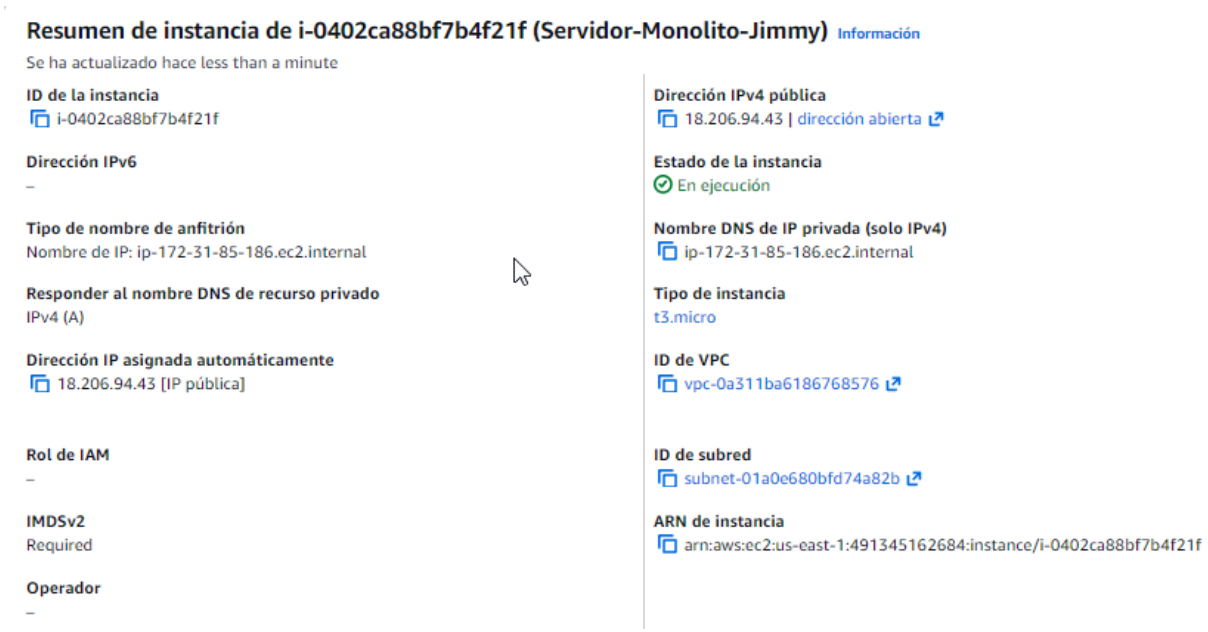
- **Problema:** La empresa usa un único servidor on-premise que sufre caídas en picos de tráfico y tiene altos costos de mantenimiento.
- **Solución:** Migrar a una arquitectura monolítica escalable en AWS usando **EC2, ALB, ASG y CloudWatch**.
- **Objetivo:** Asegurar escalabilidad automática y alta disponibilidad.

2. Paso a Paso de la Implementación.

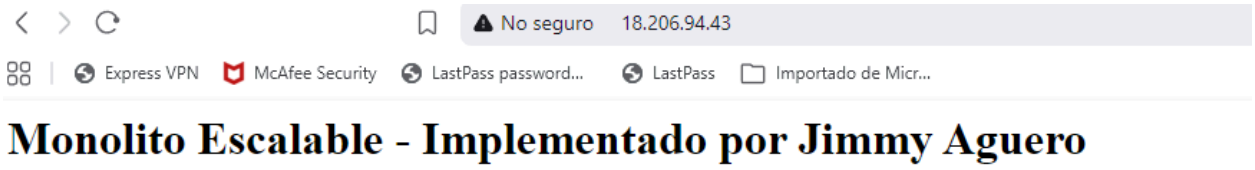
a. Instancia Base (EC2)

Se realizó el lanzamiento de una instancia Amazon EC2 (`t2.micro`) utilizando la AMI de Amazon Linux 2023. Para la configuración del servidor, se utilizó un script de *User Data* que automatiza la instalación y el inicio del servidor web Apache (httpd). Como evidencia, se adjunta la captura de la instancia en estado 'Running' y el acceso exitoso vía IP pública, donde se visualiza el mensaje personalizado del sitio.



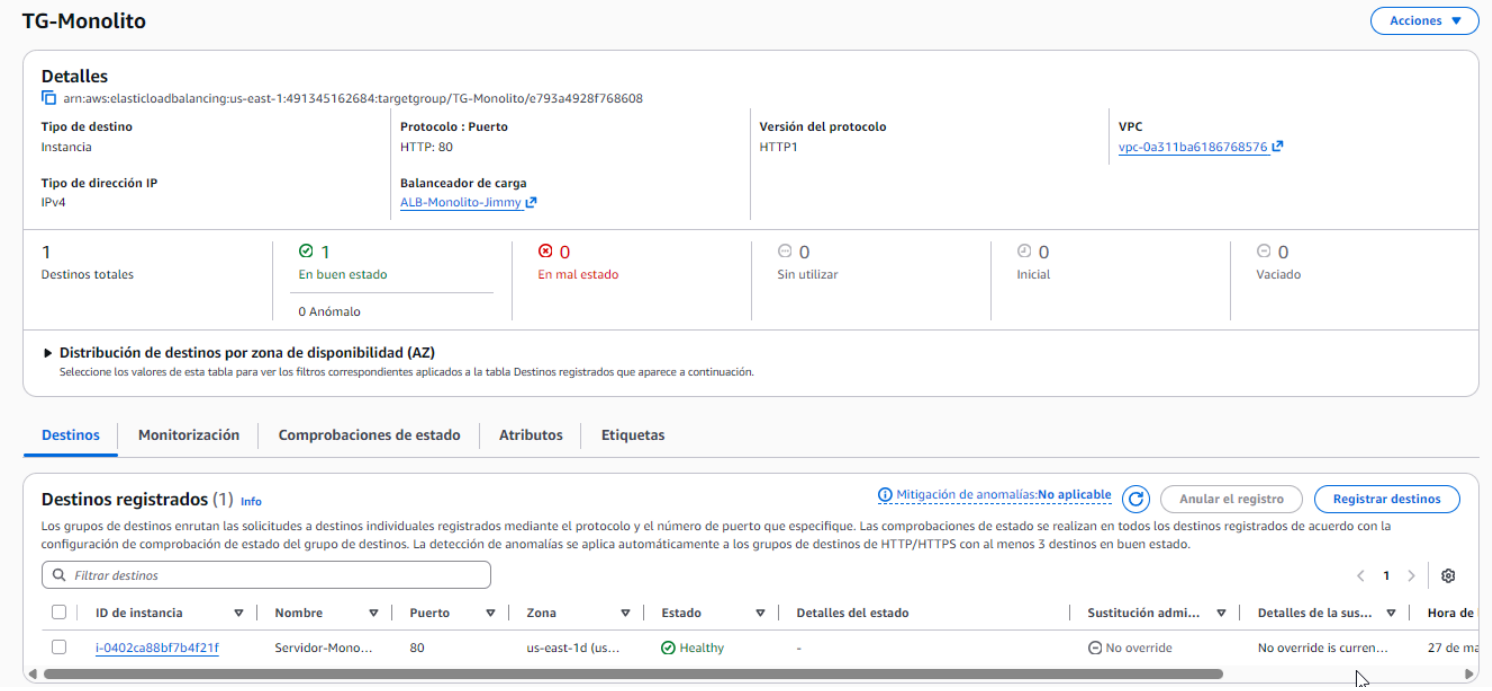


Acceso a través de IP Pública.



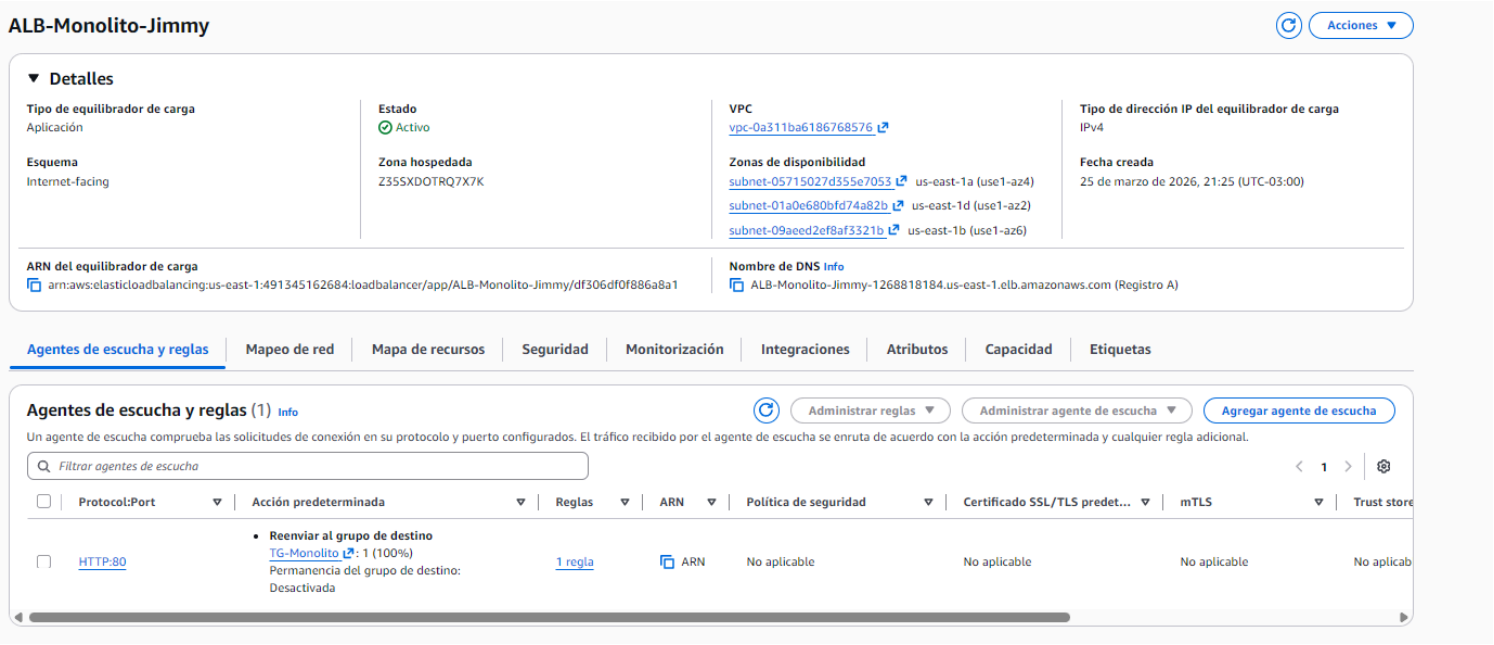
b. Target Groups (TG)

Se configuró un **Target Group** de tipo 'Instances' operando bajo el protocolo HTTP en el puerto 80. Este componente actúa como la unidad lógica que agrupa las instancias del monolito, encargándose de realizar **Health Checks** constantes mediante peticiones a la ruta raíz (`/`). Estos chequeos aseguran que el Application Load Balancer solo derive tráfico a nodos que respondan con un código de estado 200 OK, garantizando así que los usuarios finales nunca se encuentren con un servicio caído.



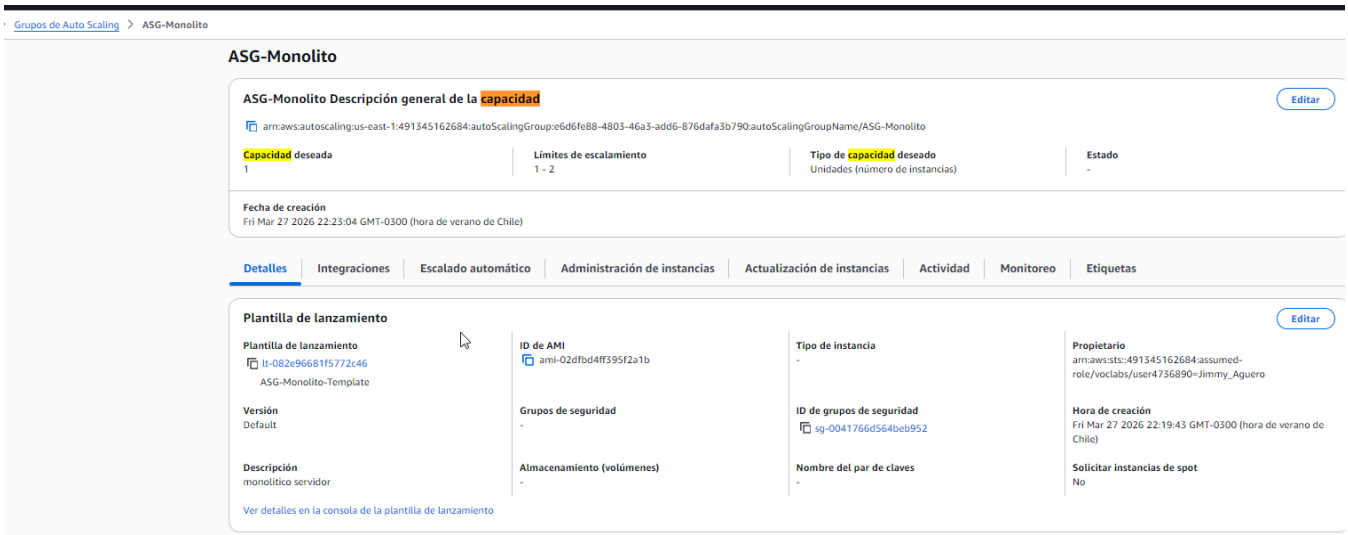
c. Load Balancers (ALB)

Para garantizar la alta disponibilidad y distribuir el tráfico entrante, se configuró un Application Load Balancer (ALB). Este recurso se vinculó a un Target Group que monitorea la salud de las instancias en el puerto 80. En las capturas se observa la configuración del *listener* y la validación final del acceso a la aplicación utilizando el DNS proporcionado por AWS, confirmando que el balanceador redirige el tráfico correctamente.



d. Auto Scaling (ASG)

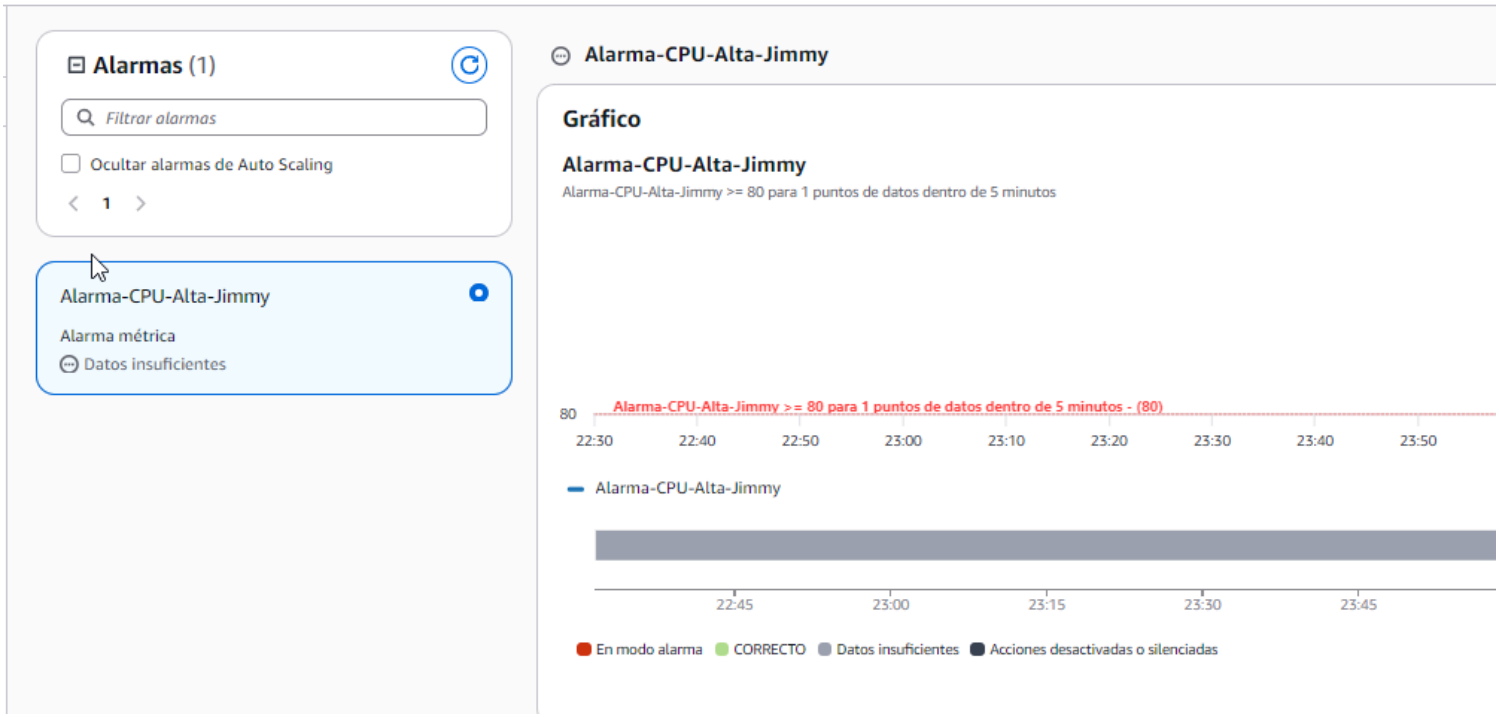
Con el fin de responder a la demanda de forma elástica, se implementó un Auto Scaling Group (ASG) a partir de una plantilla de lanzamiento (*Launch Template*). Se definió una capacidad deseada de 1 instancia y una máxima de 2. Esta configuración asegura que, ante un fallo o aumento de tráfico, el sistema sea capaz de auto-sanarse o escalar horizontalmente de manera automática sin intervención manual.



e. Monitoreo y Alarma de CloudWatch

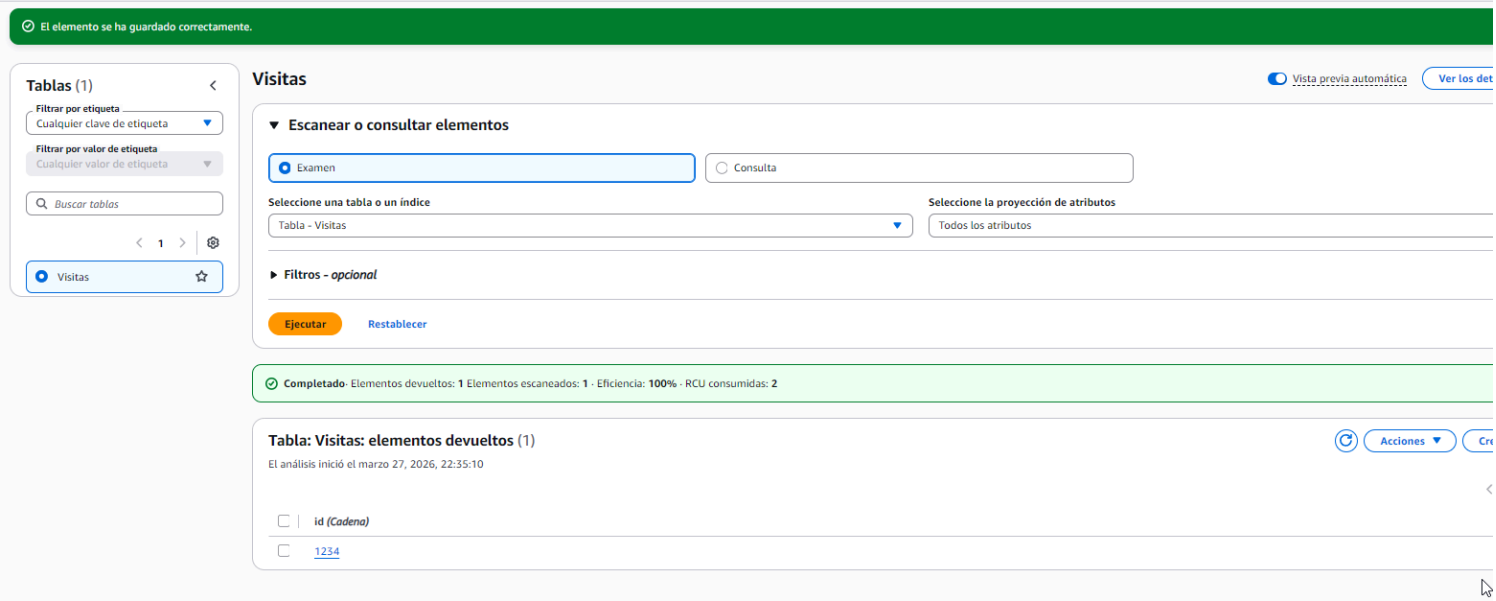
Se estableció una estrategia de monitoreo mediante Amazon CloudWatch, creando una alarma basada en la métrica de utilización de CPU. La alarma fue configurada para activarse al superar el 80% de consumo, disparando la política de escalado del ASG. La captura adjunta muestra la alarma en estado 'OK', indicando que el sistema se encuentra bajo supervisión constante y listo para reaccionar ante picos de carga

Alarma-CPU-Alta-Jimmy

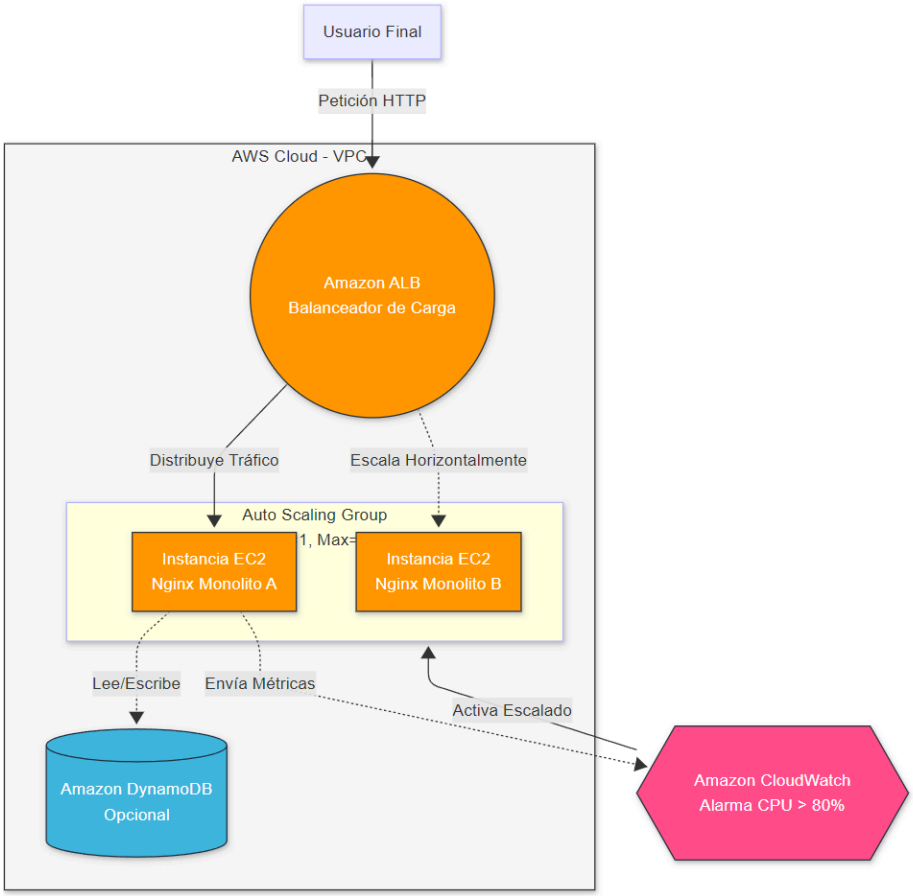


f. DynamoDB

Para separar la capa de datos del servidor de aplicación, se creó una tabla en Amazon DynamoDB denominada 'Visitas'. Se configuró una clave primaria (`id`) de tipo String y se realizó la inserción manual de un ítem para validar la persistencia NoSQL. Esta implementación demuestra cómo el monolito puede evolucionar hacia un modelo donde los datos residen en un servicio gestionado y altamente disponible.



3. Diagrama de Arquitectura



4. Conclusiones y Lecciones Aprendidas

Para cerrar tu reporte, te sugiero incluir estos puntos:

- **Escalabilidad:** El uso de ASG permite que la empresa solo pague por los recursos que necesita en tiempo real.
- **Disponibilidad:** El ALB distribuye el tráfico, asegurando que si una instancia falla, el servicio no se interrumpa.
- **Monitoreo:** CloudWatch permite reaccionar de forma proactiva ante problemas de rendimiento.
- **Lección Aprendida:** Tuve problemas de visibilidad del sitio usando el Balanceador de carga, ya que ingresaba directamente en el servidor usando la IP (EC2) pero al pasar a través del Balanceador de carga no podía llegar arrojando **error 503** verificando los *Health Checks* y la configuración del servidor web, se pudo resolver que los grupos de seguridad tenían distinta zona configurada, al alinear las zonas pude entrar a través del Balanceador de carga sin ningún problema..